T01.S04 — Measurement Discipline

Topic: T01 — Real-Time Programming Foundations | **Depends on:** T01.S02, T01.S03 **Status:** expanded | **Assignments:** A001 | **Est. theory time:** 1 session

> The question this subtopic answers: *how do I take a timing measurement that is actually true?* This is a short subtopic with a long shadow — `T13` is built on the habits formed here, and every performance claim you make for the next two years is only as good as this.

Theory

U01 — Clocks: which one, and why the others are wrong

What it is. Picking the right time source, and knowing what each one guarantees.

Why it exists. C++ gives you three standard clocks with different guarantees, and two of them will produce wrong results in a game loop.

The mechanism.

clock	monotonic?	use for
`std::chrono::steady_clock`	**yes**	**Everything in the game loop.** Guaranteed never to go backwards.
`std::chrono::high_resolution_clock`	unspecified	Nothing. It is an alias for one of the other two, implementation-defined. Avoid.
`std::chrono::system_clock`	**no**	Wall-clock timestamps for logs and saves only. NTP or a user can move it backwards mid-frame.

Use `steady_clock`, store durations as `int64` nanoseconds or as `double` seconds, and convert once at the boundary.

Two subtleties worth knowing:

- **Resolution vs precision.** `steady_clock::period` tells you the representable tick, not the actual resolution. On Linux it is typically nanosecond-representable with ~20–50 ns real granularity; on Windows `QueryPerformanceCounter` is ~100 ns. Measure it (`A001` requires this).
- **`rdtsc` and TSC.** Reading the CPU timestamp counter directly is faster and finer-grained, but requires an invariant TSC, is not guaranteed synchronized across cores, and is not a wall-clock. Real profilers use it (`T13.S02`). Do not use it for the game loop.

Where it bites. `system_clock` in a game loop: the day a user's NTP sync steps the clock backwards, `dt` is negative and the simulation explodes. It is rare, it is real, and it is a five-second fix now.

Read.

- `[CPPREF]` — the `<chrono>` clock pages, particularly the guarantee table.
- `[GEA3]` ch. 8.5.1–8.5.2 — the engine view of time and clock choices.

Check yourself.

1. Why is `high_resolution_clock` the wrong default despite the name?
2. What guarantee does "steady" give you, and give a scenario where its absence causes a visible bug.

U02 — Measuring without perturbing the measurement

What it is. The set of ways a timing measurement can be about itself.

Why it exists. Timers cost time, optimizers delete code that has no observable effect, and caches make the first run unrepresentative. Each of these can dominate the thing you are trying to measure.

The mechanism.

- **Timer overhead.** A `steady_clock::now()` pair costs on the order of tens of nanoseconds. Fine for a whole frame; nonsense for a function called 100k times per frame. Measure the overhead explicitly (time an empty pair, N times) and know your floor. `A001` requires reporting it.
- **Dead code elimination.** If you compute something and don't use it, `-O2` removes it and you measure an empty loop. Consume the result: accumulate it into a `volatile`, or use a `DoNotOptimize`-style barrier (as Google Benchmark provides).
- **Warm-up.** The first iterations pay for cold i-cache, cold d-cache, cold branch predictors, and possibly page faults on first touch. Discard a warm-up run and say how many you discarded.
- **Frequency scaling and thermals.** CPU boost clocks decay under sustained load. A 10-second benchmark can be 15% slower at the end than the start, and if you compare A-then-B you attribute that to B. Fix: interleave A and B, or pin the governor to `performance`.
- **Other processes.** A browser rendering behind your game moves the numbers. Run benchmarks on a quiet machine and *say* what the machine was.
- **Run-to-run variance.** One run is not a measurement. Report the median of ≥5 runs and the spread. If your claimed improvement is smaller than your run-to-run spread, you have not measured an improvement.

Where it bites. All of them, silently. There is no error message for "you measured the wrong thing"; the number just comes out and it looks reasonable. The only defence is a checklist, which is why this unit is a checklist.

Read.

- Google Benchmark's user guide — https://github.com/google/benchmark/blob/main/docs/user_guide.md — read the sections on `DoNotOptimize` and on why it discards results; the design is the lesson.
- `[AGNERFOG]` "optimizing_cpp" §16 — on measurement pitfalls.

Check yourself.

1. You time a function at 0 ns. Give three distinct explanations.
2. Why does comparing "run A, then run B" bias against B on a laptop?

U03 — Histograms, percentiles, and what to report

What it is. The reporting format. Following directly from `T01.S02.U04`: distributions, not means.

Why it exists. Because the shape of the distribution *is* the user experience, and a mean discards the shape entirely.

The mechanism. For frame times, collect every frame into a preallocated ring buffer (never allocate in the loop) and report:

```
frames: 3600   window: 60.0 s   config: RelWithDebInfo   vsync: off
                            machine: <cpu>, <gpu>, governor=performance

  mean    9.84 ms   (101.6 FPS equivalent)
  p50     9.61 ms
  p95    12.40 ms
  p99    15.82 ms
  p99.9  38.10 ms   <-- 3.6 frames in this run; investigate
  max    62.30 ms
  1% low  76.2 FPS   (mean of the slowest 36 frames, reported as FPS
                      because that is the convention consumers know)
```

Percentiles from a sorted copy of the samples are fine at this scale (3600 samples sorts in microseconds). For always-on in-game measurement where you cannot store every sample, use fixed-width bucket histograms (e.g. 0.25 ms buckets up to 50 ms, plus an overflow bucket) — O(1) per sample, no allocation, and percentiles are accurate to the bucket width. That is what `A001` builds.

Always report alongside the numbers: build config, vsync state, machine, and what the game was doing. A frame time without a workload description is not a measurement, it is a number.

Where it bites. Percentile interpolation methods differ, and people compare p99s computed differently. Pick one (nearest-rank is simplest: the value at index `ceil(p/100 * n) - 1` in sorted order), state it, and be consistent.

Read.

- Gil Tene "How NOT to Measure Latency" — https://www.youtube.com/watch?v=lJ8ydIuPFeU — if you skipped it in `T01.S02`, watch it here.
- HdrHistogram's rationale — https://hdrhistogram.github.io/HdrHistogram/ — the bucketing design, which is what you are building a simplified version of.

Check yourself.

1. Why is a fixed-bucket histogram preferable to storing all samples for an always-on in-game profiler?
2. What does "1% low FPS" mean precisely, and how does it relate to p99 frame time?

U04 — Your first profiling pass

What it is. The method, in the small. `T13.S02` does this properly; this unit is the version you need to not waste `A001` and `A002`.

Why it exists. So that "it's slow" becomes a question with a procedure attached rather than a prompt for guessing.

The mechanism.

1. **Reproduce, with a fixed workload.** A specific scene, a specific input sequence, a specific duration. If the workload varies, nothing after this step means anything.
2. **Establish the baseline.** Numbers, in the format from U03. Write them down in the assignment's `NOTES.md`. You cannot claim an improvement against a remembered number.
3. **Decide CPU or GPU first.** These are different investigations. Crude but effective test: drop the resolution to 1/4. If frame time improves substantially you are GPU/fill-bound; if it barely moves you are CPU-bound. (`T13.S02` has the proper method.)
4. **Form a hypothesis before you look.** Write it down. This matters: it makes you notice when the data disagrees, which is the useful case.
5. **Measure at coarse granularity first.** Which of the frame's phases (from `T01.S01.U01`) owns the time? Only then zoom in.
6. **Change one thing.** Re-measure with the identical workload. Report median-of-5 and the spread.
7. **Record the result even when it was zero.** "SoA made no difference at n=200" is a real finding and stops you retrying it in six months.

Tools on Linux, in order of how soon you'll want them: `perf stat` (cheap overview: cycles, instructions, cache misses, branch misses), `perf record` / `perf report` (sampling profiler, no instrumentation needed), then Tracy (https://github.com/wolfpld/tracy — a frame profiler built for games; you will end up here at `T13`).

Where it bites. Skipping step 4. Without a written hypothesis you tend to find what you expected in noisy data, and the whole exercise becomes a confirmation ritual.

Read.

- Brendan Gregg on `perf` — https://www.brendangregg.com/perf.html — the reference for the Linux tooling.
- `[GEA3]` ch. 8.5.3, and skim ch. 4 for where the time goes in a real frame.

Check yourself.

1. Describe the resolution test for CPU vs GPU bound, and give a case where it gives a misleading answer.
2. Why write the hypothesis down before measuring, rather than after?

Implementation

assignment	size	target rung	what it forces you to understand
A001	XS	R2	Clock selection, timer overhead, allocation-free sampling, and percentiles from a bucketed histogram

Order and overlap. `A001` is done here and immediately reused: its histogram becomes `A002`'s on-screen overlay, `A006`'s loop instrumentation, `A007`'s allocator benchmark harness, `A016`'s broadphase benchmark, and the seed of `A042`'s real profiler. Six uses of one hour's work.

If you are short on time. Do `A001` in full; it is one hour. The unit you cannot skip within it is the histogram — storing all samples works at `A001`'s scale and stops working exactly when you need it most.

Discussion Log

Open Questions

Connections

- `T01.S02` — the reasons behind the reporting format this subtopic specifies.
- `T13.S02` — profiling done properly, with the tools this unit introduces.
- `A042` — where `A001`'s histogram becomes a real frame profiler.